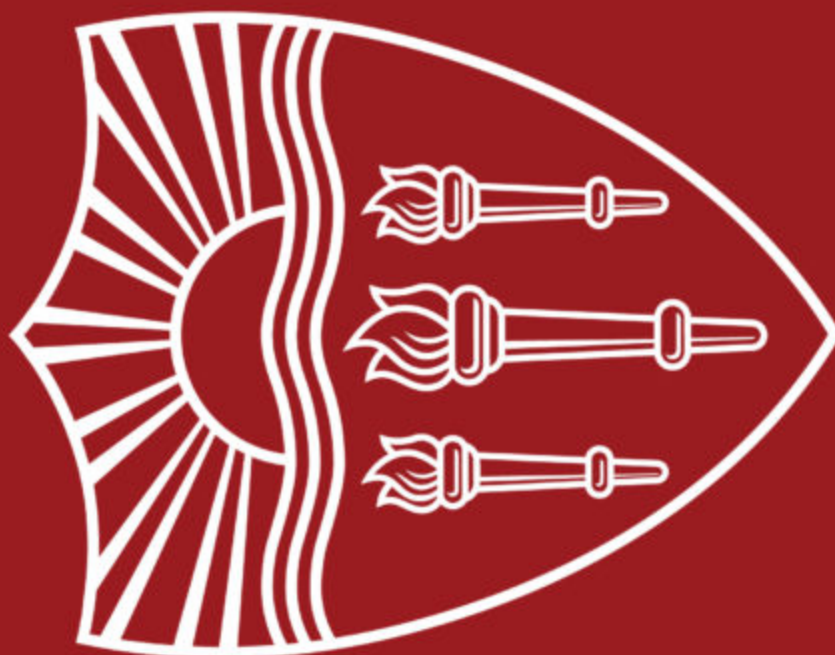




Lecture 15: (Pre)Training LLMs

Instructor: Swabha Swayamdipta

USC CSCI 444 NLP

Oct 29, 2025



Announcements + Logistics

- HW2 grades out
- Today: Project Proposal Due
- Next Monday: Quiz 4
- Next Wednesday: Paper discussions on three classic papers:
 - Direct Preference Optimization: DPO;
 - InstructGPT;
 - Low Rank Adaptation: LoRa;
 - Read these before next Wednesday, okay to not fully understand all the details, we will discuss these in class
- Special Request: Research Opportunity in my lab / USC Center for AI and Society for someone with experience in front-end development.

Lecture Outline

- Announcements
- Recap: Modern Generation via Sampling
- Large Language Models
 - What does training involve?
 - LLM pretraining data
 - LLM pretraining recipes
- Interacting with LLMs: prompting

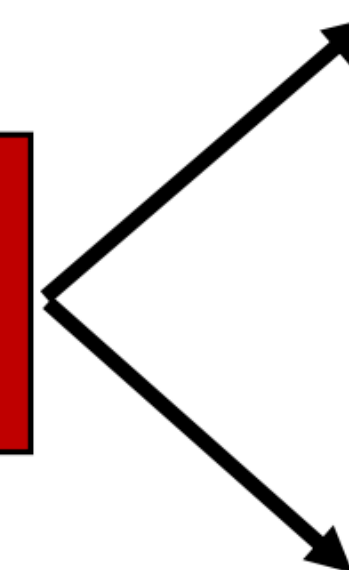
Recap: Modern Generation with Sampling

Pure / Ancestral Sampling

- Sample directly from P_t
 - Similar to sampling from an n -gram language model
- Still has access to the entire vocabulary
- But if the model distributions are of low quality, generations will be of low quality as well
- Often results in ill-formed generations
 - No guarantee of fluency

$$y_t \sim P_t(w) = \frac{\exp(S_w)}{\sum_{v \in V} \exp(S_v)}$$

He wanted
to go to the



restroom
grocery
store
airport
bathroom
beach
doctor
hospital
pub
gym

Top- K Sampling: Value of K

- Only sample from the top K tokens in the probability distribution
 - Common values are $K = 50$

He wanted
to go to the



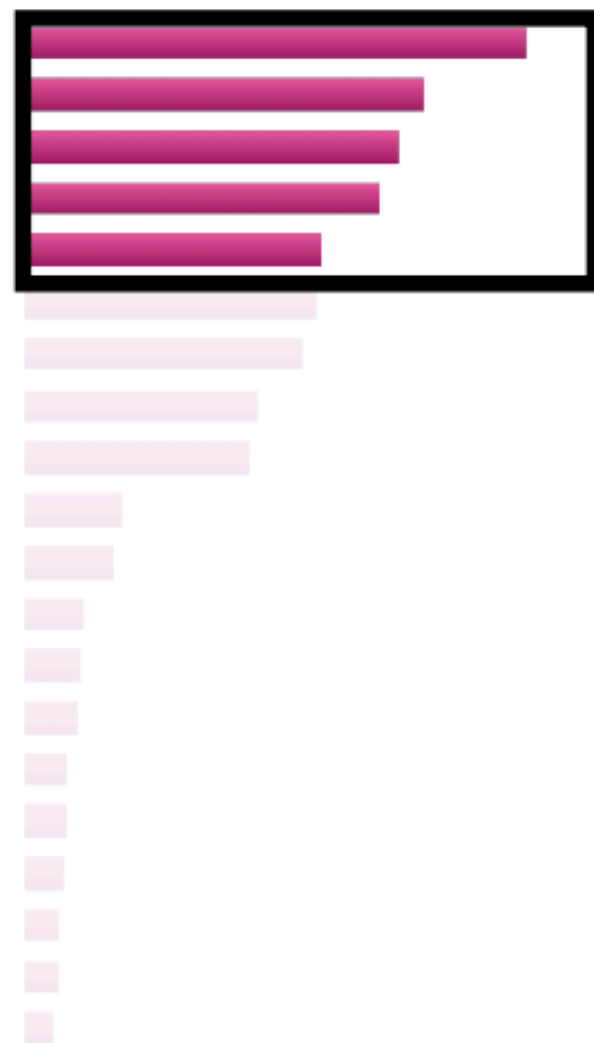
restroom
grocery
store
airport
bathroom
beach
doctor
hospital
pub
gym

- Increase K yields more diverse, but risky outputs
- Decrease K yields more safe but generic outputs

Nucleus (Top- P) Sampling

- Solution: Top- P sampling
 - Sample from all tokens in the top P cumulative probability mass (i.e., where mass is concentrated)
 - Varies K depending on the uniformity of P_t

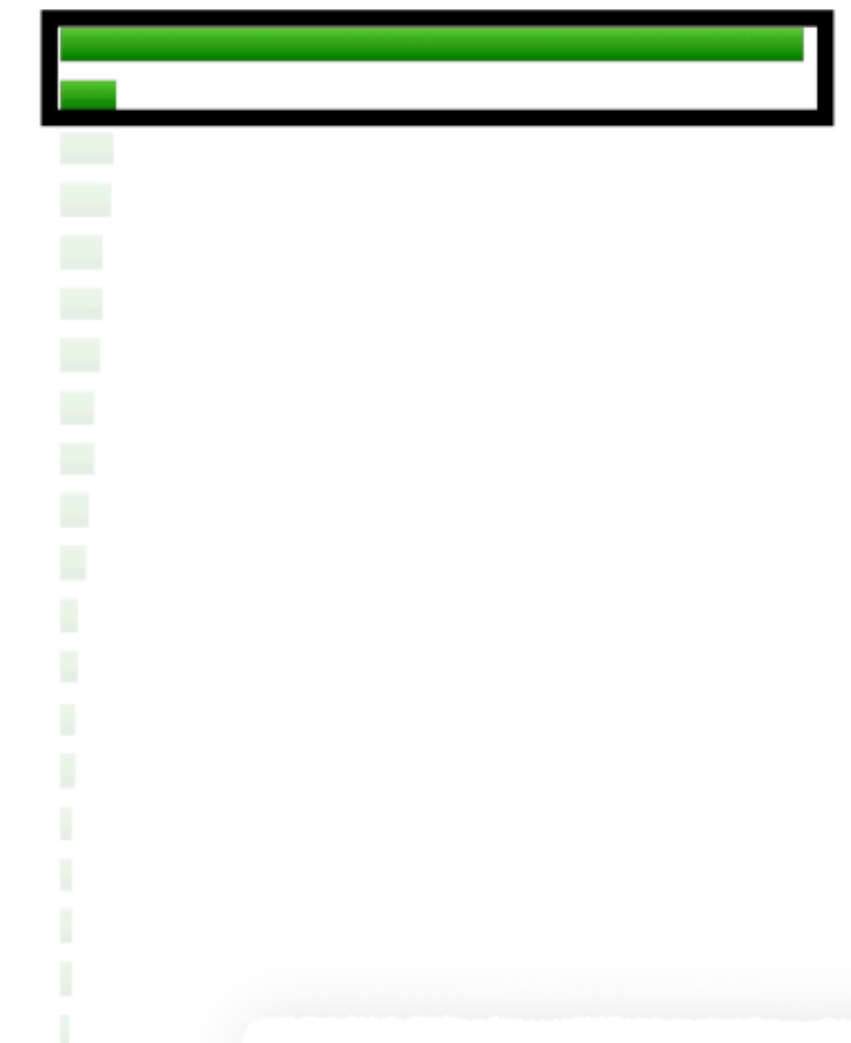
$$P_t^1(y_t = w \mid \{y\}_{<t})$$



$$P_t^2(y_t = w \mid \{y\}_{<t})$$



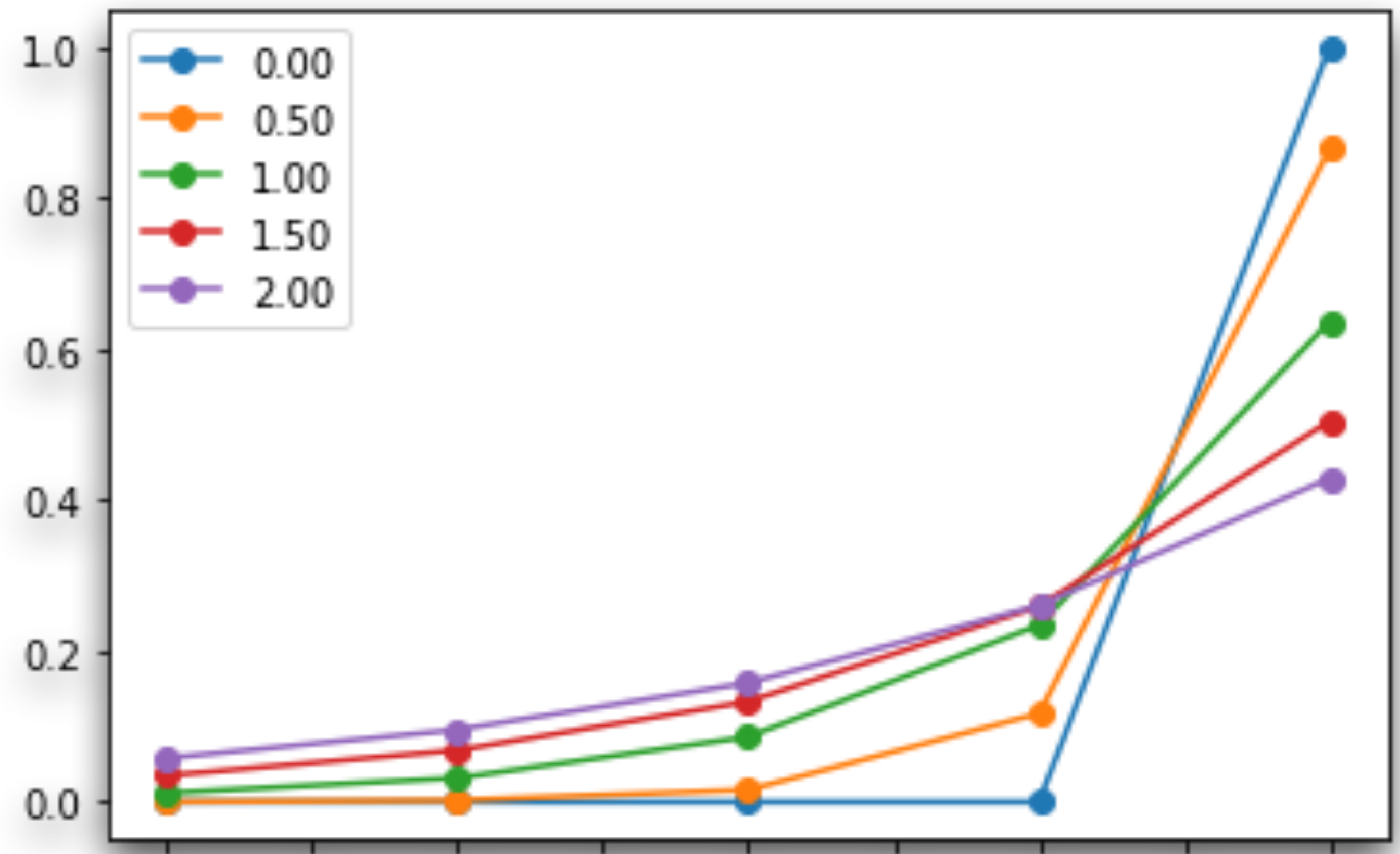
$$P_t^3(y_t = w \mid \{y\}_{<t})$$



Temperature Scaling

$$P(y_t = w) = \frac{\exp(S_w/\tau)}{\sum_{v \in V} \exp(S_v/\tau)}$$

- Raise the temperature $\tau > 1$: P_t becomes more uniform
 - More diverse output: probability is spread around vocab
- Lower the temperature $\tau < 1$: P_t becomes more spiky
 - Less diverse output: probability is concentrated on top words



Comparing different decoding algorithms

- Generate text to continue a given context
 - Open-ended generation
- Same decoding algorithms are also useful for close-ended generation tasks



WebText

So what's new in my life?

09/11/18 - Just got back from vacation.

Beam Search, $b=16$

Just got back from vacation.

09/10/18 - Just got back from vacation. Just got back from vacation.

09/09/18 - Just got back from vacation. Just got back from vacation.

09/08/18 - Just got back from vacation. Just got back from vacation.



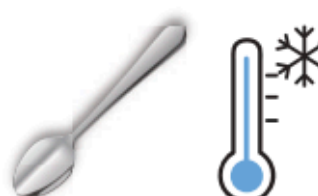
Pure Sampling

01/05/19 - Visited The Subaru Canada Heritage 15th Star for my vehicle research.

That was a wonderful experience. Thanks again for your knowledge and support.

05/23/19 - I was thrilled to receive my Subaru Legacy character! It fits me perfectly and makes me feel like a kid again!

06/15/19 - Just regained my licence for my different coloured 2016 WRX / STI...

Sampling, $t=0.9$

Just got back from vacation.

09/07/18 - Happy 9th birthday to me! Happy 9th birthday to me!

09/01/18 - Sofwidden also! Sofwidden also!

08/25/18 - Happy birthday to me! Happy birthday to me!

08/17/18 - Night shift! ☐☐Night shift! ☐☐

08/17/18 - Lucky me! Lucky me!

Top-k, $k=640$

Just got back from vacation.

09/03/18 - After more than 40 years, I might have an apartment in Virginia Beach.

After more than 40 years, I might have an apartment in Virginia Beach.

08/20/18 - Going for a hike at Mount Eerie in Northeast Virginia Spring

Going for a hike at Mount Eerie in Northeast Virginia Spring

Top-k, $k=640$, $t=0.7$

Just got back from vacation.

09/08/18 - I've gotten really sick. - I've gotten really sick.

09/07/18 - My wife and I are getting married in February.

- My wife and I are getting married in February.

09/06/18 - I'm so excited to go back to college this fall.

- I'm so excited to go back to college this fall.

Nucleus, $p=0.95$

Just got back from vacation.

07/12/18 - Happy birthday to Swingu, who is nearly 5 years old. I would like to say hi to him on the road as well as when I ride with him. You cannot go to work without feeling physically sick or psychologically exhausted because you can barely breathe. Even if you ride on rollercoaster even once, it is easy to recover from the physical side of it.

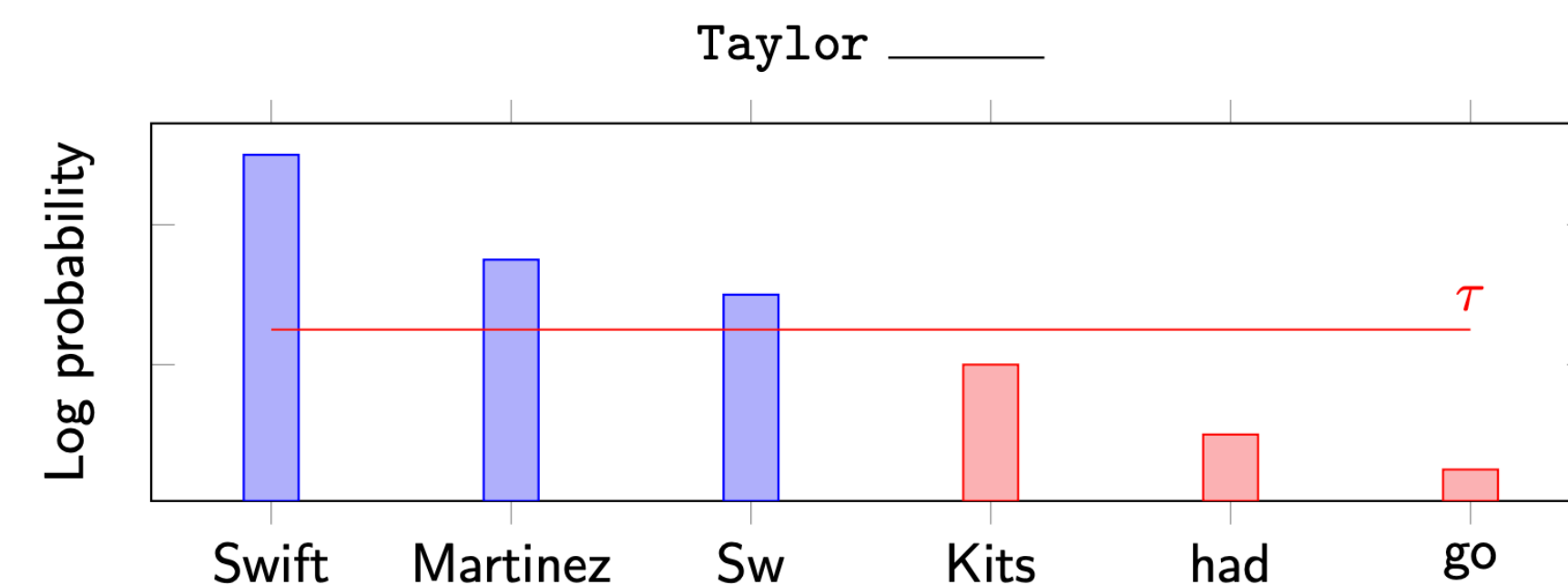


WebText

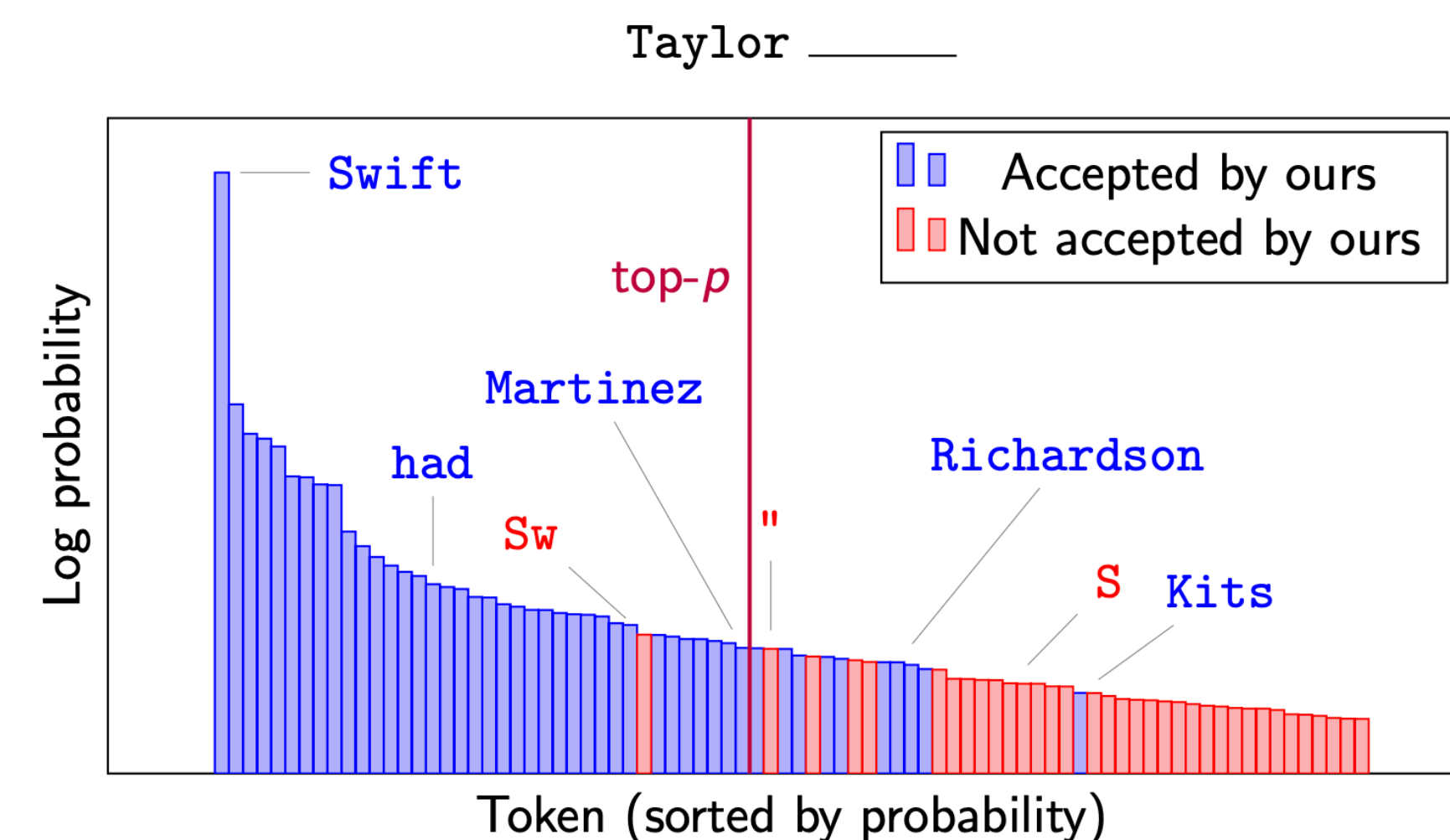
I just got back from a much needed and really great nine day vacation to my remote Arizona property. It was a really restful and relaxing visit. I got a lot accomplished while I was there, but still found time to just goof off and have fun too. I got to do some astronomy, even though the weather was pretty cloudy most of the time. Here is a 50 minute exposure of M101. It turned out pretty good.

Truncation-based Sampling

- Nucleus Sampling is an example of truncation sampling
 - Certain properties of language models (mismatch between vocabulary size and hidden dimensionality) make threshold sampling a great choice!
 - [Finlayson, H., K., Swayamdipta, S., ICLR 2024]
- Locally-Typical Sampling: Similar to Nucleus Sampling, but based on conditional entropy (entropy of a distribution determines its randomness) [Meister et al., 2022]
- η -Sampling: Entropy dependent threshold that also takes into account absolute probabilities [Hewitt et al., 2022]
- BAT Sampling: More flexible than truncation [Finlayson, H., K., Swayamdipta, S., ICLR 2024]

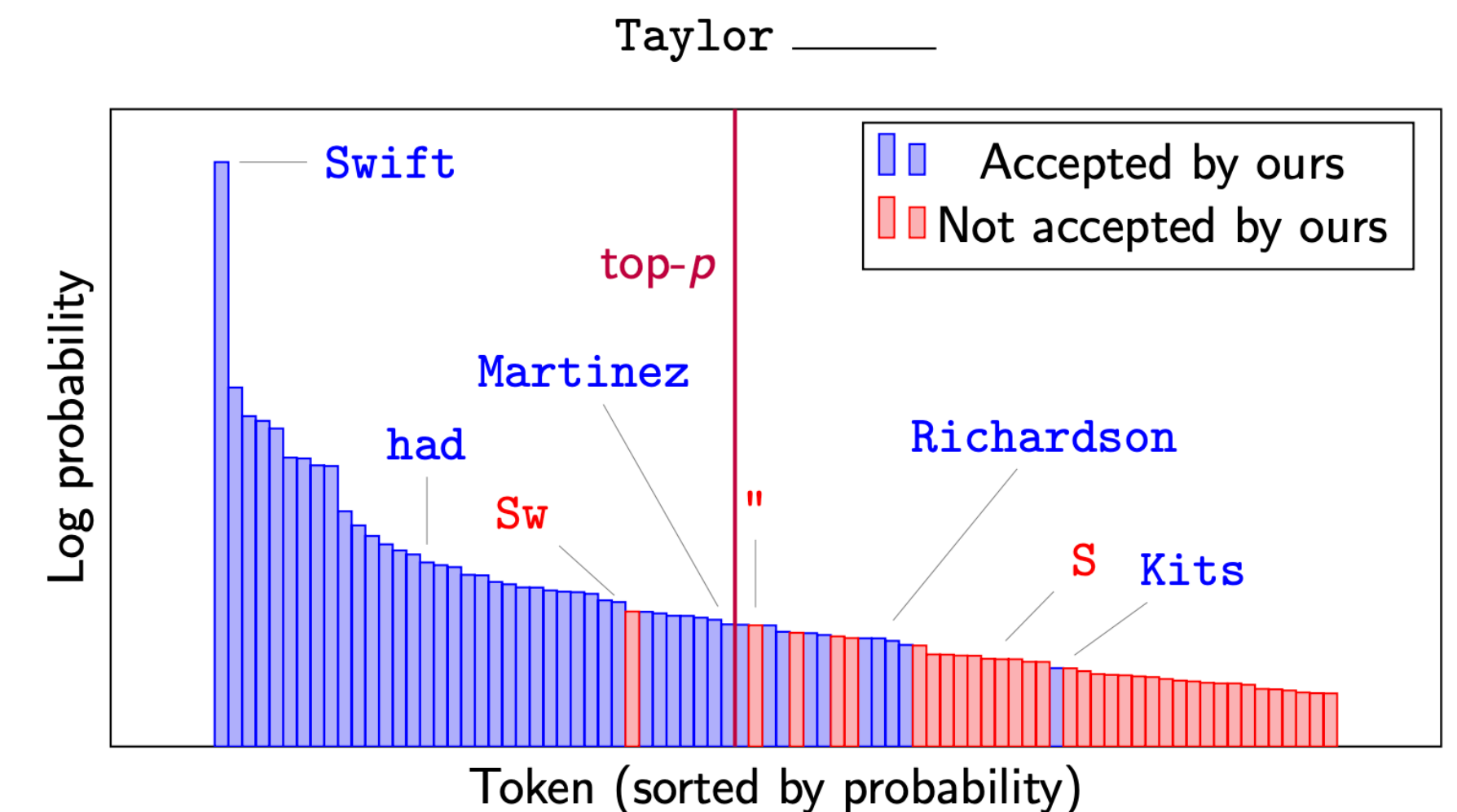


Choose a threshold τ and only sample tokens with probability greater than τ .



Modern Decoding: Takeaways

- Natural language distributions are very peaky but the softmax function assigns probabilities to all tokens in the vocabulary
- Hence we need approaches to truncate / modify the softmax distribution
 - Ancestral, Top- k , Top- p (Nucleus), Temperature
- Some properties of the softmax function make truncation based decoding necessary



Lecture Outline

- Announcements
- Recap: Modern Generation via Sampling
- Large Language Models
 - What does training involve?
 - LLM pretraining data
 - LLM pretraining recipes
- Interacting with LLMs: prompting

Large Language Models

What are Large Language Models?

- Models with many, many, parameters
 - Deeper layers. The largest T5 model had 11B parameters. GPT-3 has 175B parameters. Llama-3 has a 405B parameter variant.
 - More context. GPT-2 has a context length of 1024 tokens, GPT-4 and Llama-2 have 8192, Llama-3 has 128K!
 - LLMs are generally trained by filling the full context window with text. If documents are shorter than this, multiple documents are packed into the window with a special end-of-text token between them.
- Models trained on many, many tokens
 - Training data size is measured in # tokens
 - Batch size of the largest GPT-3 model is 3.2M tokens

OpenAI model's version	GPT-3 (ada, babbage, curie, davinci)	GPT-3.5 (gpt-3.5-turbo, gpt-3.5-turbo-0301, text-davinci-003, text-davinci-002)*	GPT-4-8K
Context length (max request)	2,049	4,096	8,192
Number of English words	~1,500	~3,000	~6,000
Number of single-spaced pages of English text	3	6	12

Source: [Neoteric](#)

Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
Chinchilla	70 Billion	1.4 Trillion

Three Stages of Training LLMs

Stage 1

- Pre-training on large corpus of text
- Produces a **Base Language Model**
- Continued Pre-training for domain adaptation (optional; sometimes called mid-training, stage 1.5)

Stage 2

- Post-training for Task Adaptation
- Seq2Seq Instruction Tuning (Supervised Finetuning, different meaning than BERT-style classification tasks)

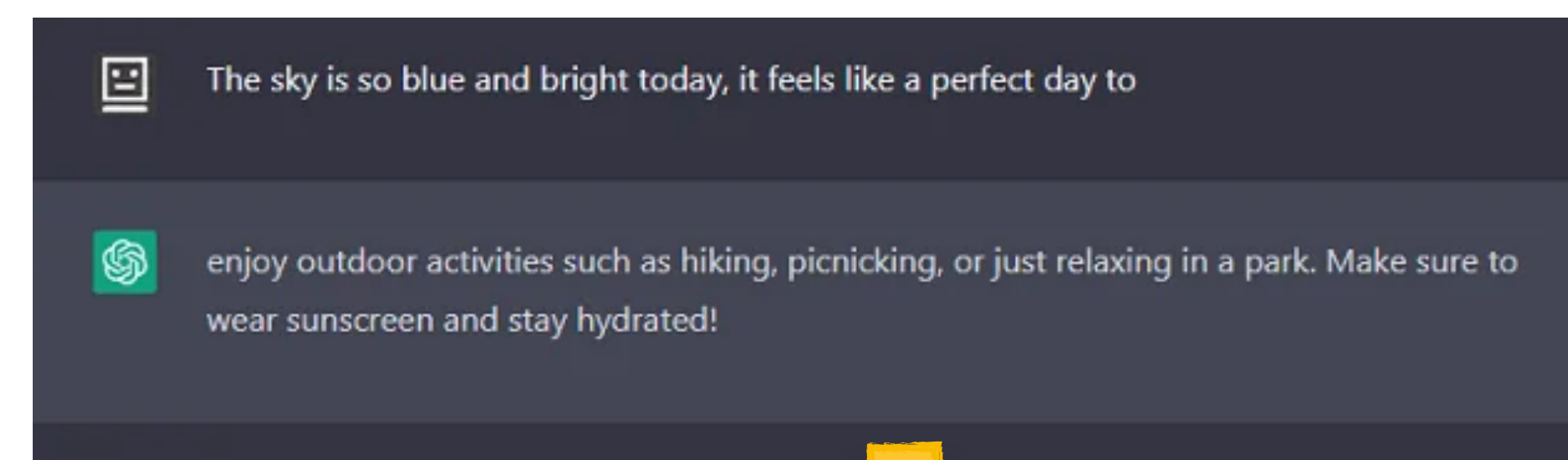
Stage 3

- Post-training for Preference Alignment
 - Either Reinforcement Learning with Human Feedback : Train a supervised classifier (reward model) on human demonstrations to provide feedback to LM. Reinforcement learning to maximize rewards given by reward model
 - Or, train with a preferred and a dispreferred generation (more popular now)

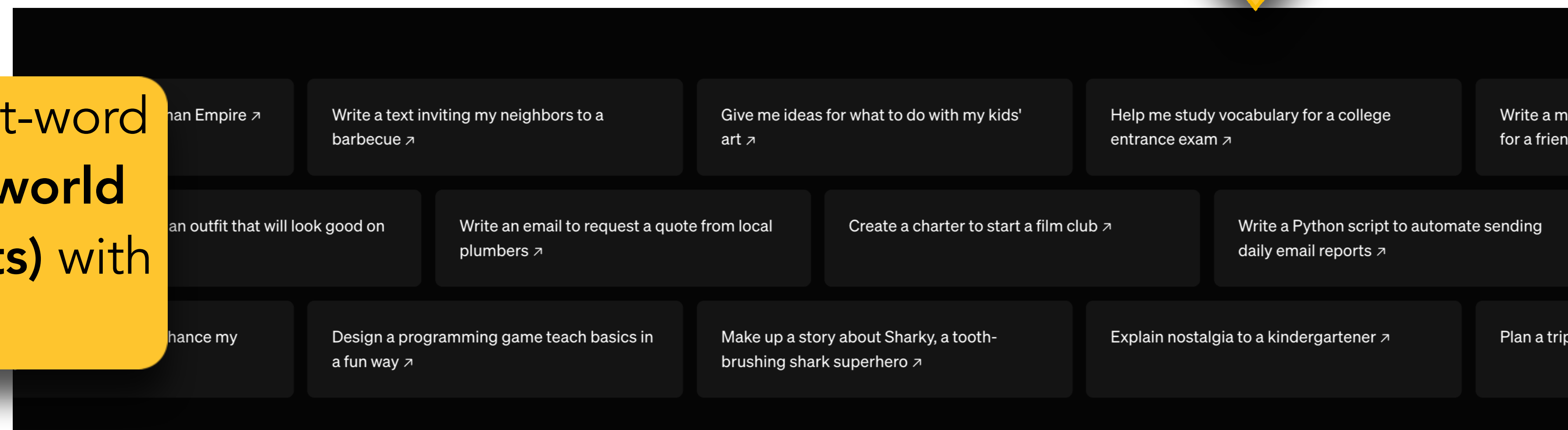
Inference : Prompting with Instructions and Demonstrations (also called examples, shots)

Pre-training and Post-training

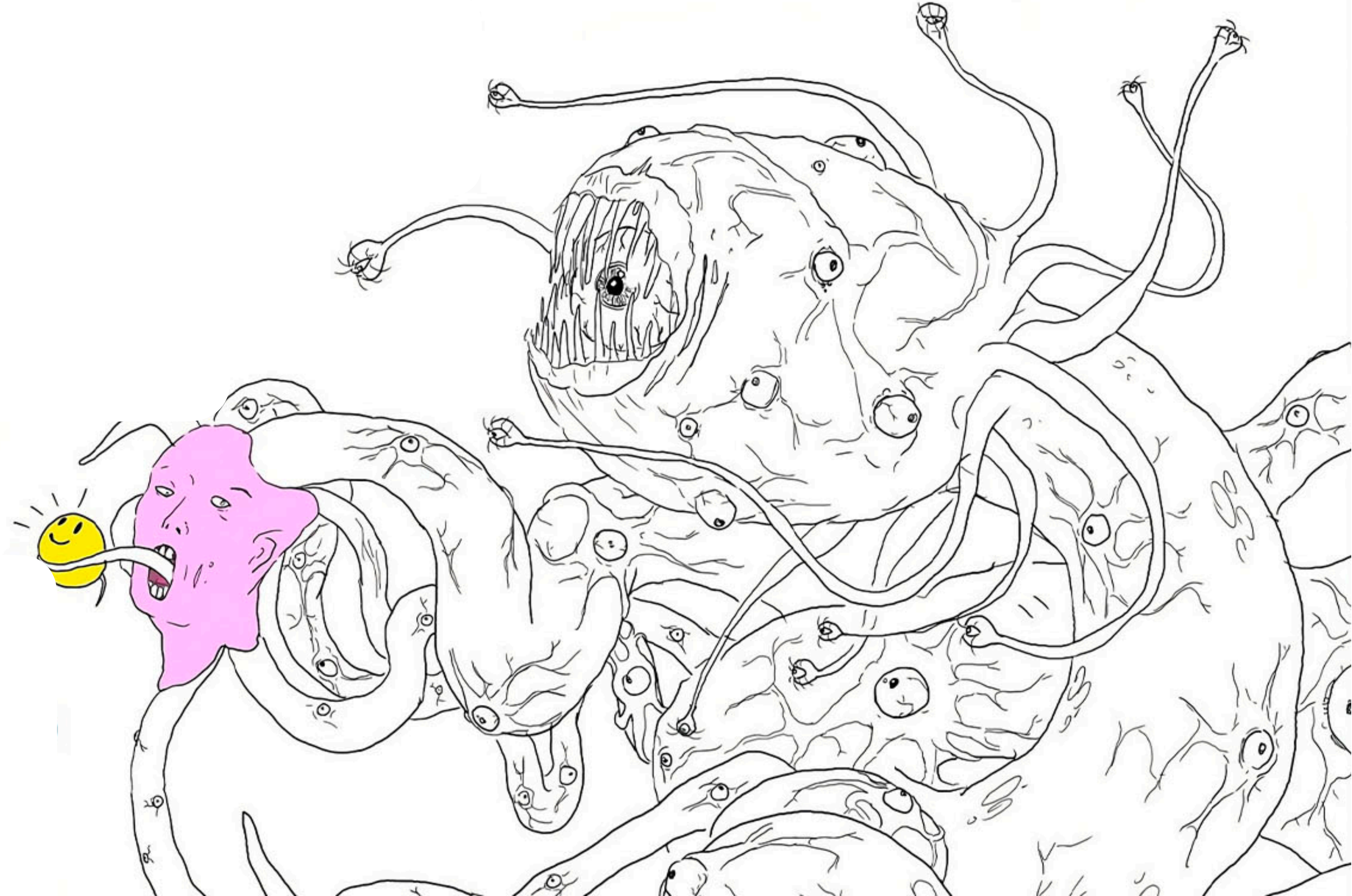
- The term “training” is no longer specific enough :)
- Pre-training: Self-supervised, standard next token prediction
- Post-training: Supervision (sequence to sequence)
 - **Instruction-Tuning**: Supervision is not necessarily via labels, but sequence pairs. Labels in standard NLP benchmarks can be converted into sequence pairs
 - **Preference-Tuning**: Collects human judgments / preferences as rewards, or a pair of preferred / dispreferred generations



Post-training converts next-word completion models into **world models (agents, assistants)** with many capabilities!



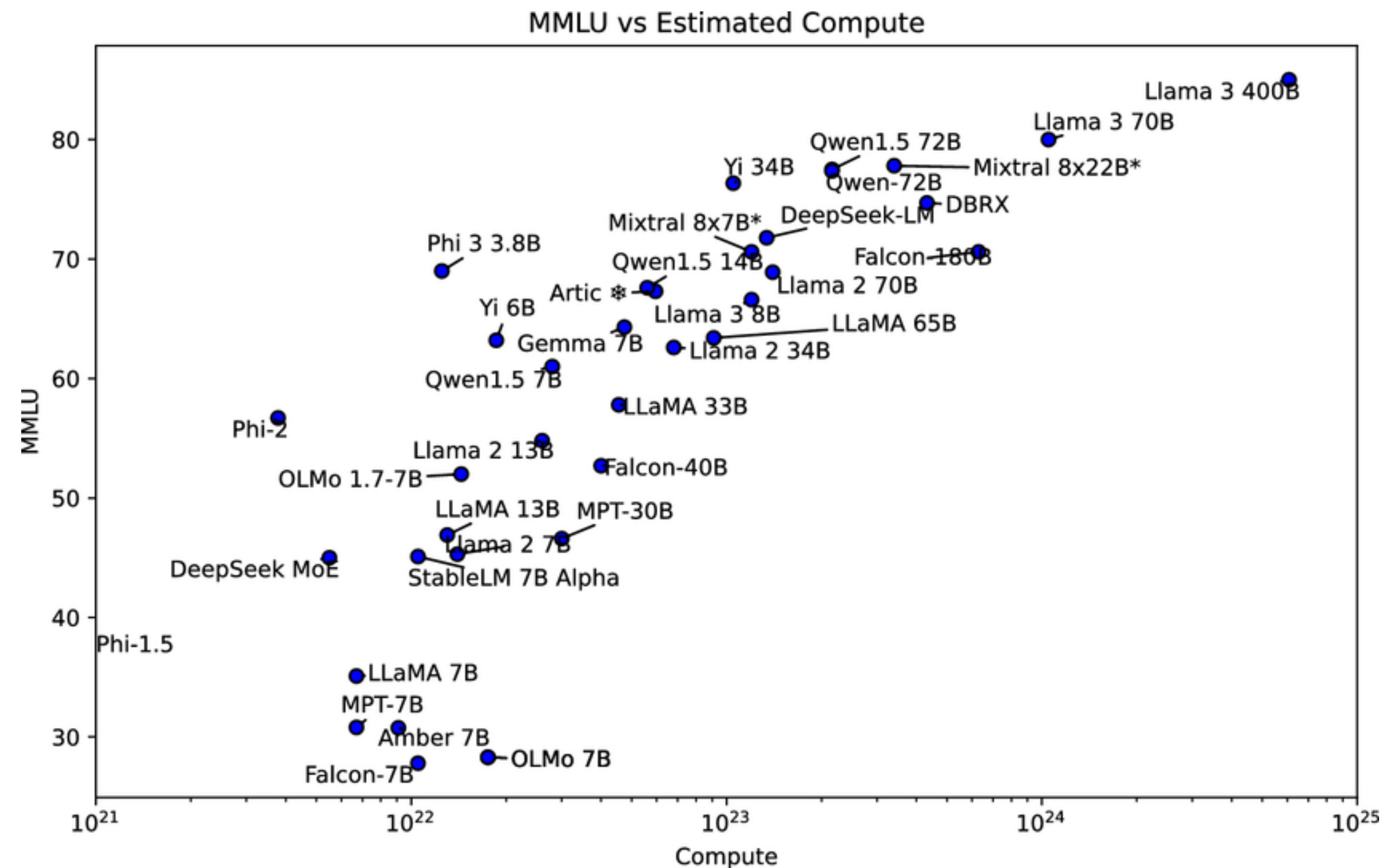
Training LLMs



Shoggoth; slide credit: Justin Cho

The Effect of LLM Scale

- Models designed with high capacity/ scale can potentially achieve high performance
- Main factors determining model scale:
 - Model architecture (Transformer vs. RNN vs. State Space Model)
 - Training FLOPs
 - # model parameters
 - # tokens of pretraining data



The Scaling “Laws” of LLMs

- Predict loss given parameter size, data size, computation

Kaplan et al., 2020 <https://arxiv.org/abs/2001.08361>

- Learn a relationship from a small number of samples
- Estimate the model behaviors at high-capacity by experimenting on low-capacity models

One example:

Chinchilla Scaling Law Formulation [1]

$$\hat{L}(N, D) \triangleq E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}.$$

L: Loss

N: # Model Parameter

D: # Training Tokens

E: A constant capturing the Entropy of the text

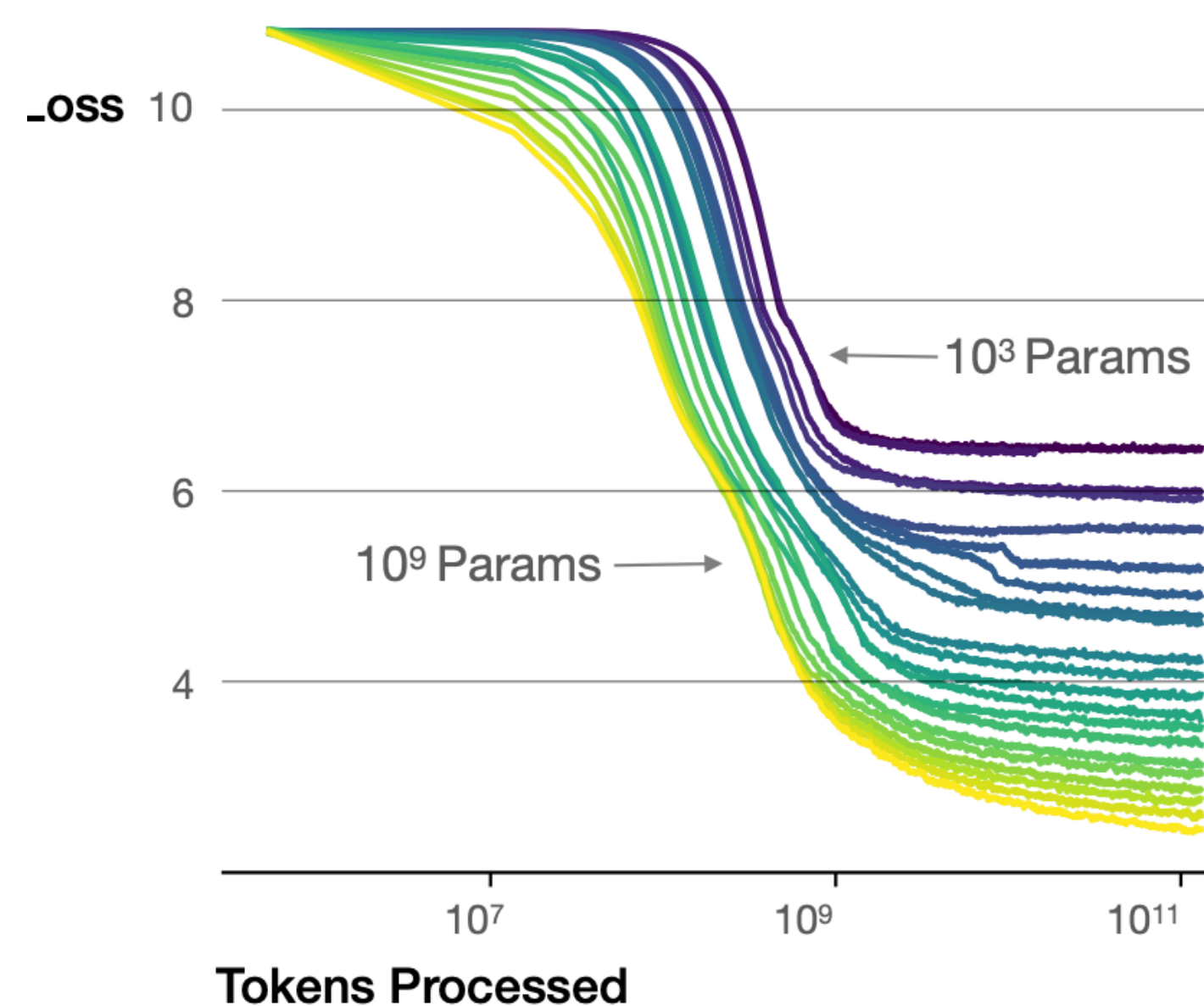
E, A, B, α , and β are to be fit during experiments

The loss of an LLM scales as a power-law with each of these properties of model training

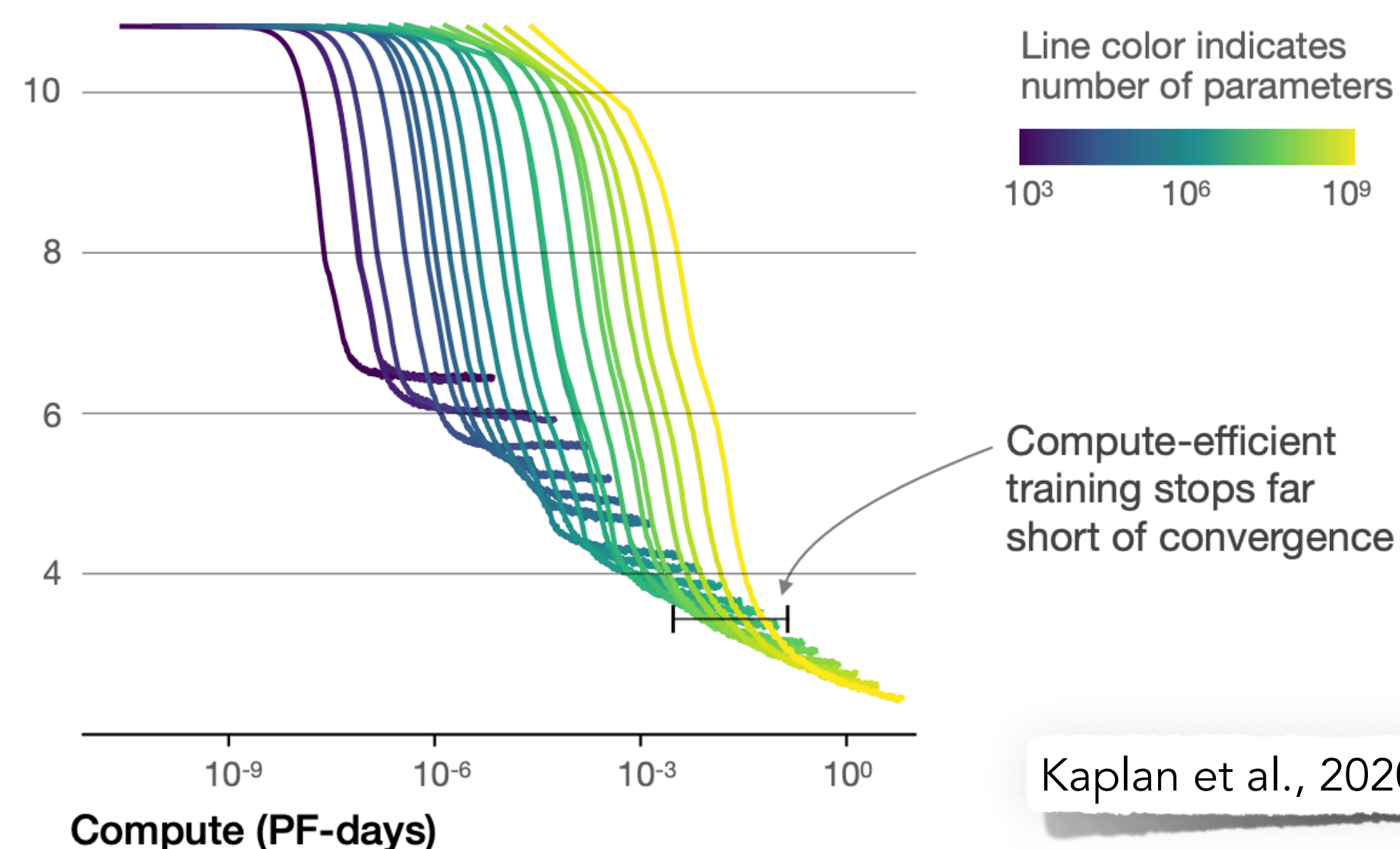
The Scaling “Laws” of LLMs

- Other architectural details such as network width or depth have minimal effects within a wide range
- Determines the optimal allocation of a fixed compute budget
 - For instance, it can help us train very large models on a relatively modest amount of data and stop significantly before convergence
 - Or smaller models on larger data (e.g. Chinchilla LM)

Larger models require **fewer samples** to reach the same performance



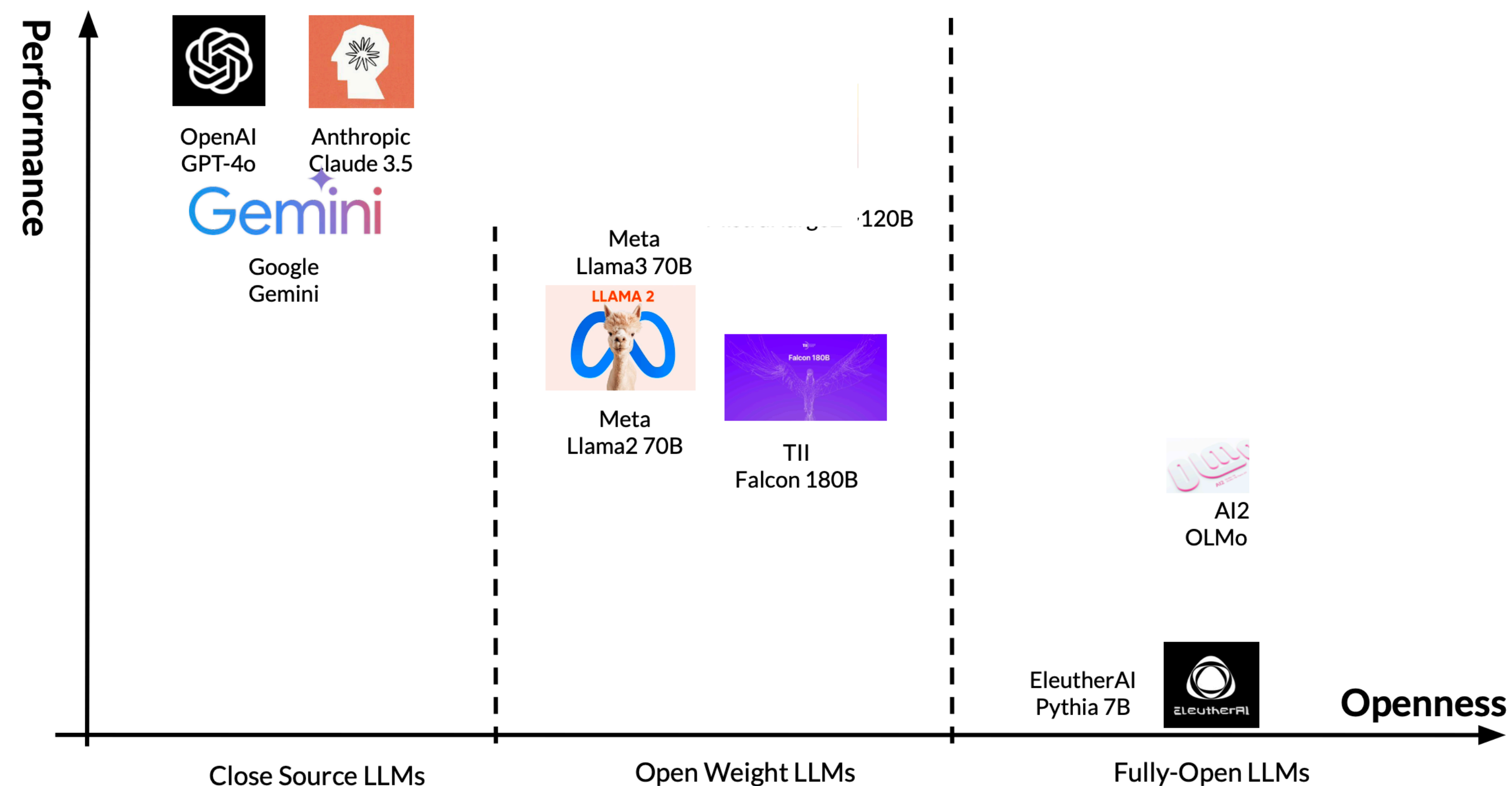
The optimal model size grows smoothly with the loss target and compute budget



Kaplan et al., 2020 <https://arxiv.org/abs/2001.08361>

Open-Source, Open-Weight vs. Closed LLMs

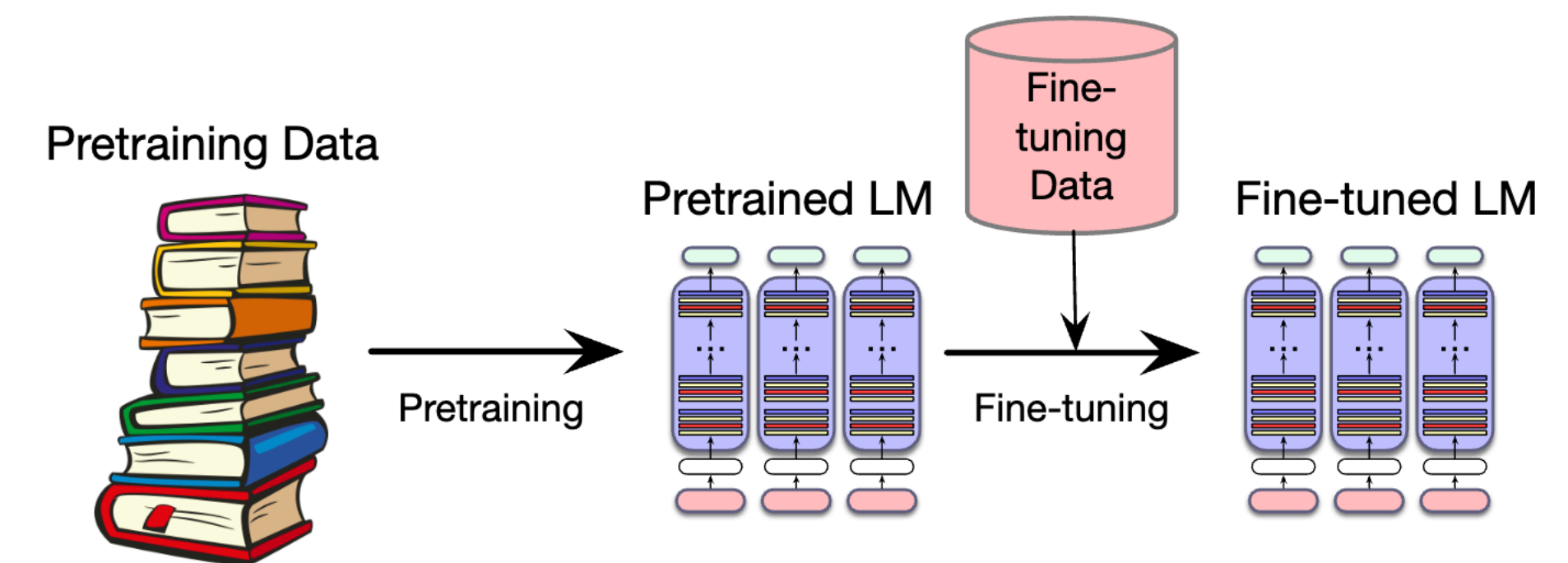
- Open-Source: everything is released. Can completely reproduce, provided you have the compute
 - e.g. Allen AI's OLMo, Eleuther AI's Pythia
 - Levels the playing field for LLM development via knowledge sharing
- Open Weight: only the final model weights are released, but no data, code etc.
 - e.g. DeepSeek-R1, Llama-3, Qwen, Mixtral
- Closed-Source / API-protected: Most popular and also, the best performing
 - e.g. GPT-4o, Claude-4.5



LLM Pretraining Data

Training Data for LLMs

- Pre- and post-training requires different kinds of data
- Pre-training needs a lot of raw text
 - Crawled from the Web: Natural Solution
 - But not all data crawled from the web is good for LM training
 - Needs to be filtered in various ways (deduplicated, removing non-natural language like code, sentences with offensive words from a blocklist).
 - Quality Filters! (Later lecture)
- Post-training / Fine-tuning
 - Instruction Tuning Data: Repurposed NLP / ML benchmarks
 - Preference Data for RLHF: Often human labels
 - Not easy to produce...



Pre-training Data

- Language models are trained on “raw text”
- To be highly capable (e.g., have linguistic and world knowledge), this text should span a **broad** range of domains, genres, languages, etc.
- A natural place (but not the only place) to look for such text is the **web**
 - Google search index is 100 petabytes; the actual web is likely even larger
 - **Private datasets** owned by big companies are even larger! [WalMart](#) generates 2.5 petabytes of data each hour!
- **Common Crawl** is a nonprofit organization that crawls the web and provides snapshots that are free to the public
 - Standard source of data to train many models such as T5, GPT-3, etc.
 - The April 2021 snapshot of [Common Crawl](#) has 320 TB
- The Colossal Clean Crawled Corpus ([C4](#)) is a larger was created to train the T5 model — 806 GB / 156 billion tokens



The Pile and DOLMA

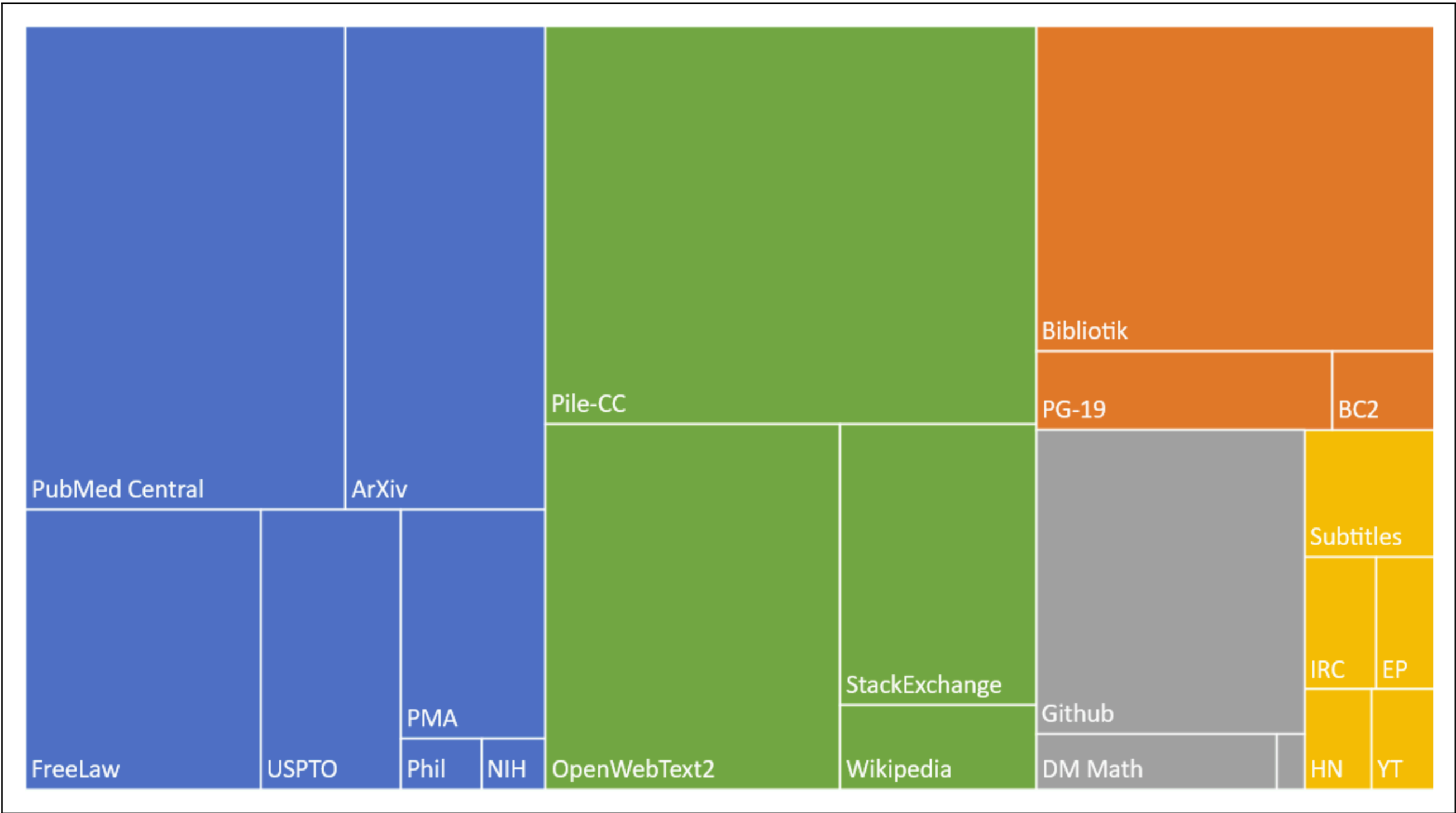


Figure 10.5 The Pile corpus, showing the size of different components, color coded as **academic** (articles from PubMed and ArXiv, patents from the USPTA; **internet** (webtext including a subset of the common crawl as well as Wikipedia), **prose** (a large corpus of books), **dialogue** (including movie subtitles and chat data), and **misc.** Figure from [Gao et al. \(2020\)](#).

The Pile: 825 GB English text corpus containing a large amount of text scraped from the web, books and Wikipedia



Source	Doc Type	UTF-8 bytes (GB)	Documents (millions)	Unicode words (billions)	Llama tokens (billions)
Common Crawl	web pages	9,812	3,734	1,928	2,479
GitHub	code	1,043	210	260	411
Reddit	social media	339	377	72	89
Semantic Scholar	papers	268	38.8	50	70
Project Gutenberg	books	20.4	0.056	4.0	6.0
Wikipedia, Wikibooks	encyclopedic	16.2	6.2	3.7	4.3
Total		11,519	4,367	2,318	3,059

Dolma is a larger open corpus of English, created with public tools, containing three trillion tokens, which similarly consists of web text, academic papers, code, books, encyclopedic materials, and social media (Soldaini et al., 2024)

Data Mixes and Deduplication

Need to mix data from different special domains

Can find data weighting empirically by performing sweeps. Also methods like DoReMi for “optimal” data mixing weights

Programming Code	Multiple Languages	Specialist Domains
• Enable model's programming ability. • Helpful in function call. • Could improve reasoning.	• Multi-lingual / non-English corpus. • Note that filtering rules that rely on language statistics might need to be adapted.	• Professional areas such as legal, medical. • Special document formats such as tables, forms, etc.

Some hypothesize that a lot of duplicate data causes the model to replace generalization with memorization

However, some duplication is needed, especially for high quality data, e.g. Wikipedia

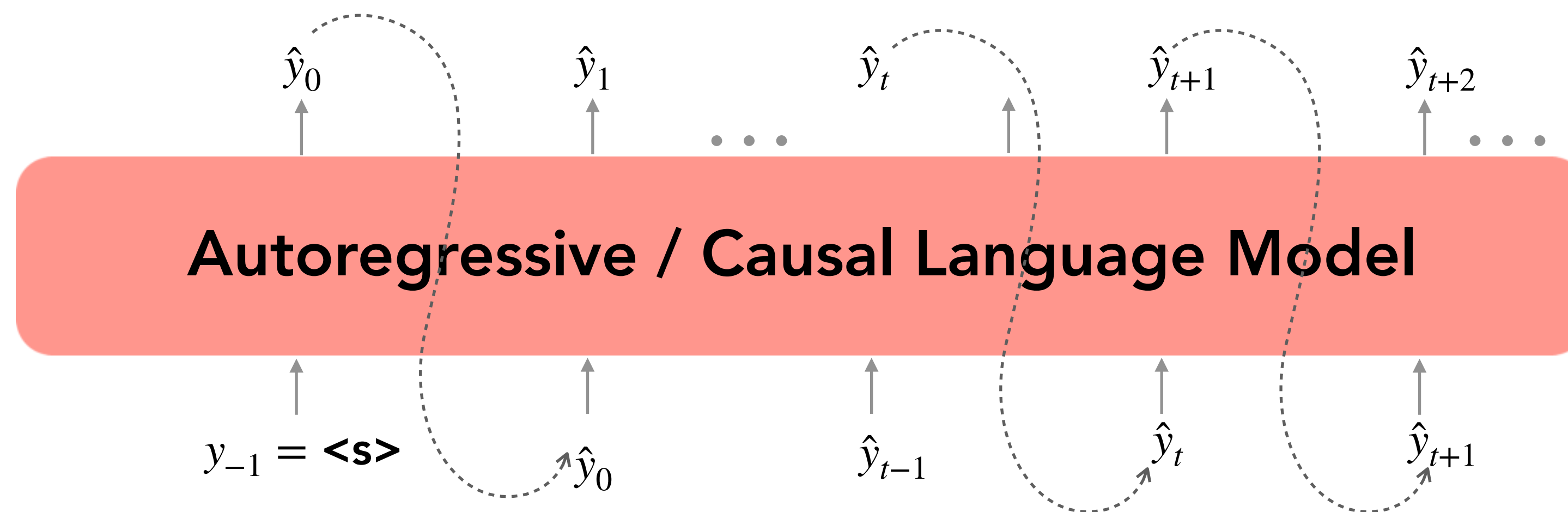
LLM Pretraining Recipes

Self-Supervised Training

At each time step t , our model takes in a sequence of tokens as input $\mathbf{y}_{<t} \in \mathbb{R}^V$ and outputs a new token, $\hat{y}_t$

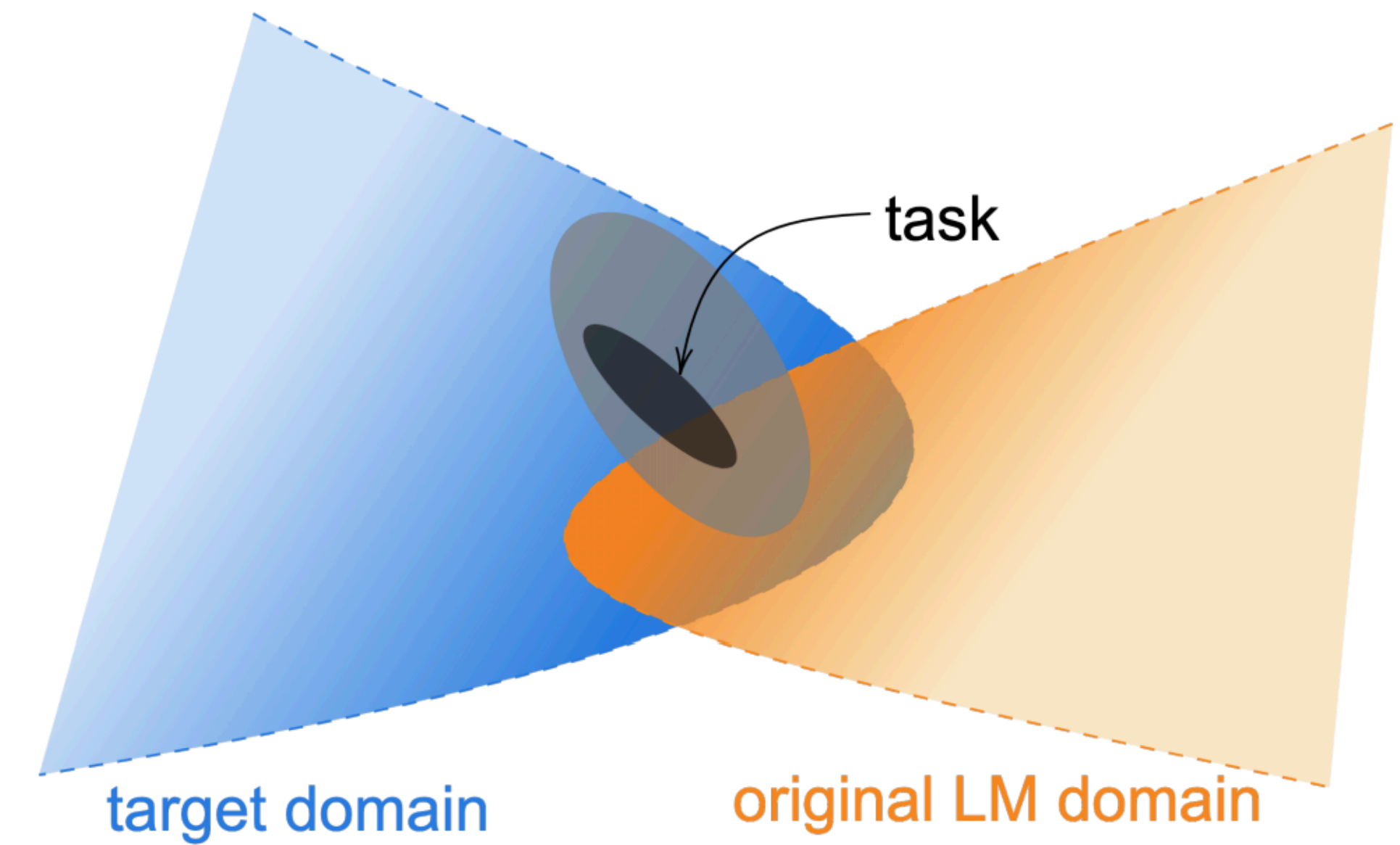
For model $f_\theta(\cdot)$ and vocabulary V , we get scores $S = f_\theta(w | \mathbf{y}_{<t})$ for every $w \in \mathbb{R}^V$

$$P(w | \mathbf{y}_{<t}) = \frac{\exp(S_{w|\mathbf{y}_{<t}})}{\sum_{v \in V} \exp(S_{v|\mathbf{y}_{<t}})}$$

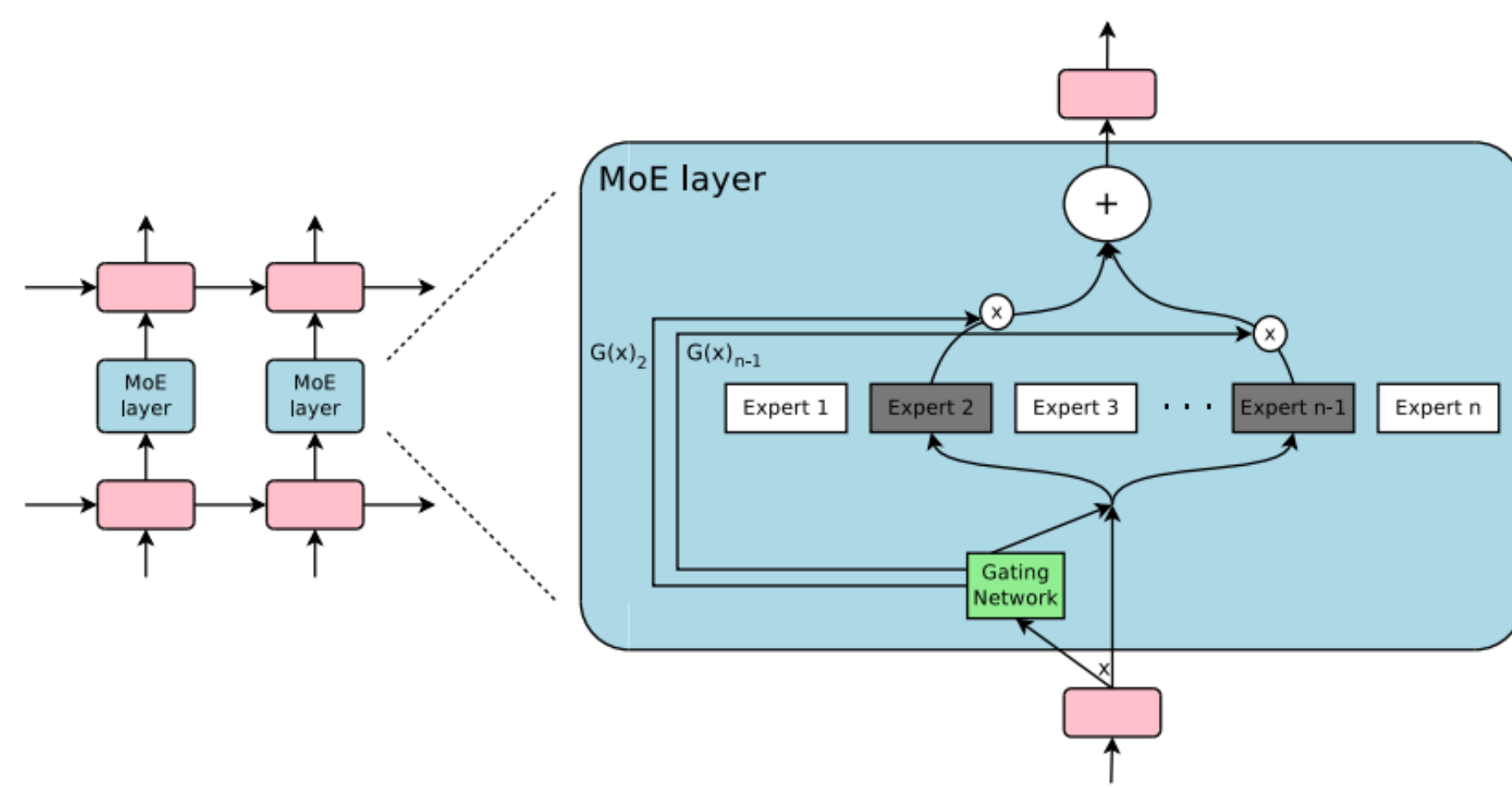


Continued Pretraining / Midtraining

- LLMs are general domain. But we still need domain-specific knowledge
 - For example, we might want a language model that works in medicine / law
 - Continue training the model on relevant data from the new domain or language (Gururangan et al., 2020)
 - Next word prediction objective
- Works better when starting from a pretrained general-domain language model, as opposed to training from scratch
- Learning rates need to be altered as the model enters this regime (see OLMo paper)



Dense LLMs vs. Sparse Mixtures of Experts



Source: Huggingface

Fedus et al., 2021. Switch Transformers

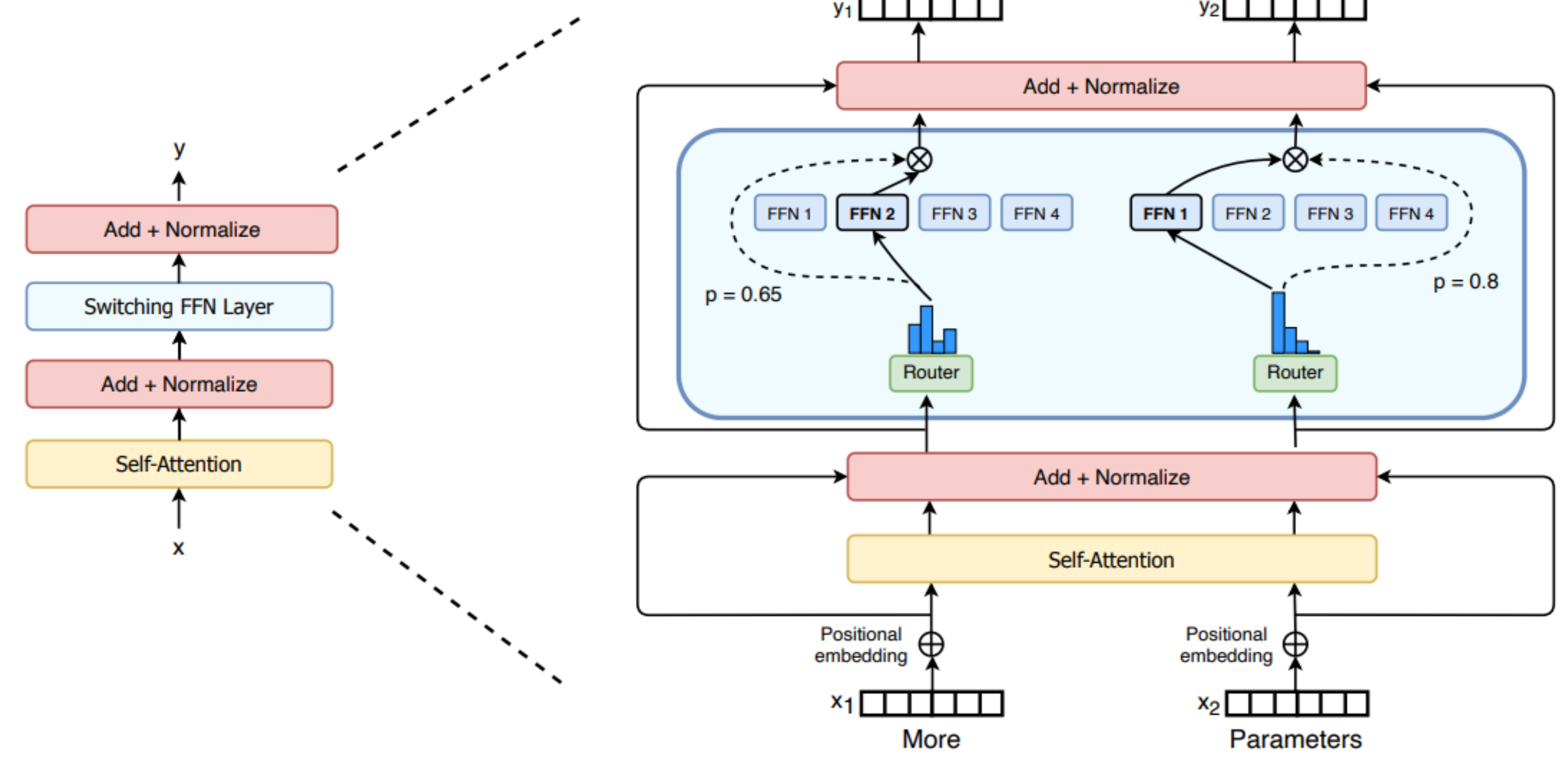


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = "More" and x_2 = "Parameters" below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

Quantization

- Quantization converts the final model into much lower precision (e.g., BF16 $\rightarrow$ Int8).

- Some parts of a model may be less suitable for quantization

Dettmers et al. 2022. [LLM.int8\(\)](#)

- Also: Quantization-Aware Training

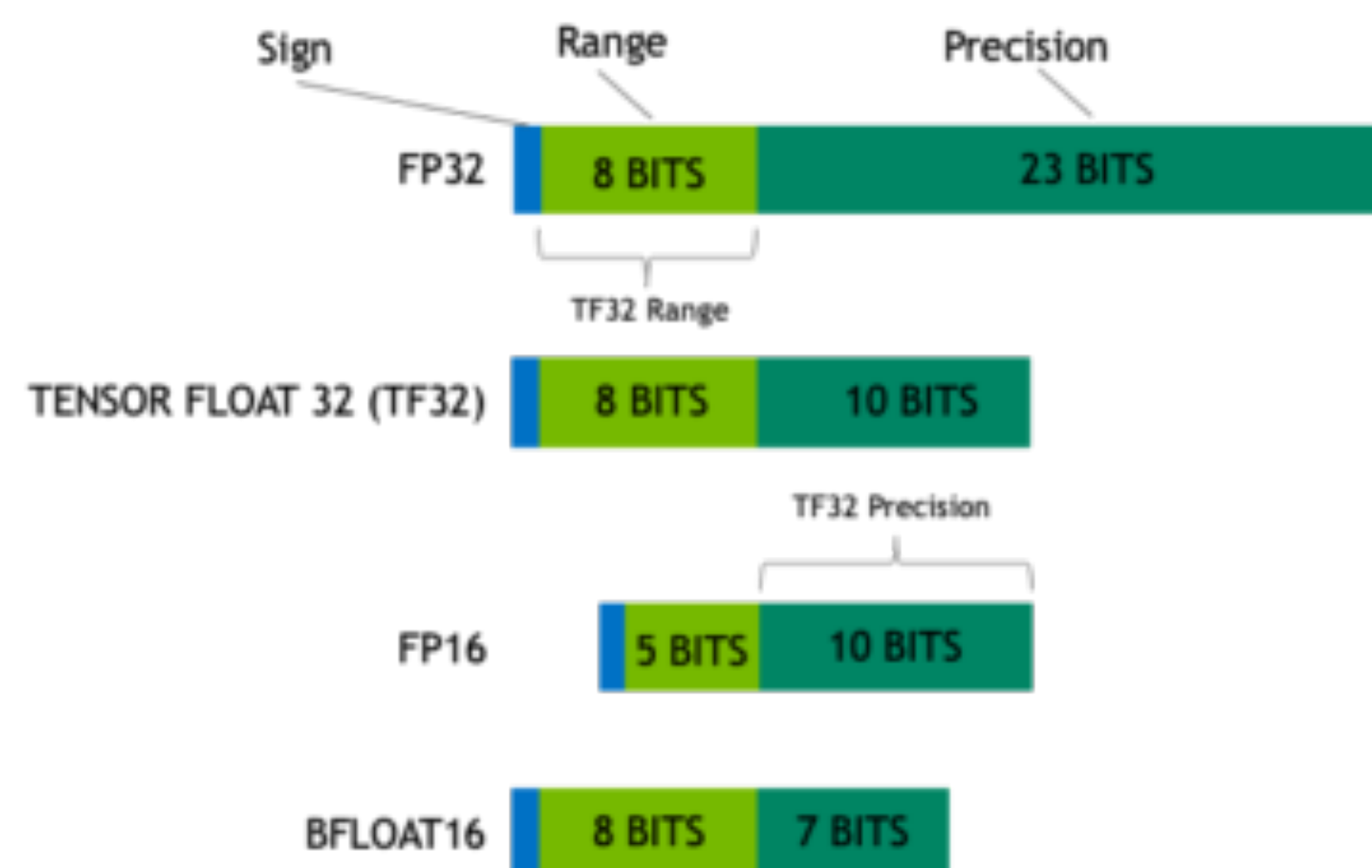


Image from [NVIDIA](#)

Comparing number formats

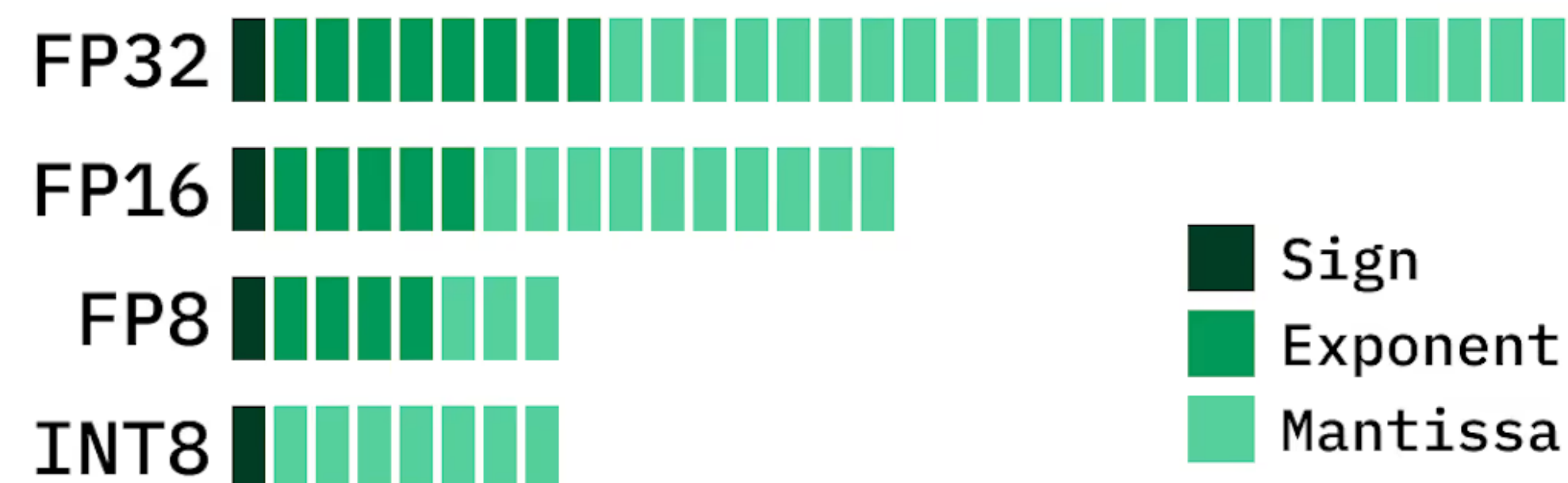


Image Source: [BaastenAI](#)